

操作系统

第三章 进程通信

并程序序 和 进程通信基本概念

同济大学计算机系

1、串程序

没有fork，没有Pthread

第2个实例： 2、matrix程序

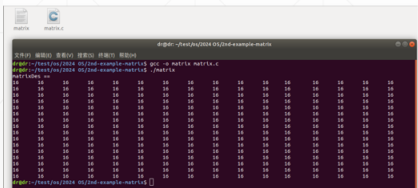
方阵自乘



```
# include <stdlib.h>
# include <stdio.h>

# define M 16
int version = 1;
int matrixOriginal[M][M];
int matrixDes [M][M];
```

./matrix



```
void produce ( int row, int column )
{
    int i;
    for (i=0;i<M;i++)
        matrixDes[row][column] += matrixOriginal[row][i] * matrixOriginal[i][column];
}

int main()
{
    int i , j;

    for (i=0;i<M;i++)
        for (j=0;j<M;j++)
            produce(i , j);

    for (i=0;i<M;i++)
        for (j=0;j<M;j++) {
            matrixOriginal[i][j] = 1;
            matrixDes[i][j] = 0;
        }

    printf("matrixDes == \n");
    for (i=0;i<M;i++) {
        for (j=0;j<M;j++)
            printf("%d\t", matrixDes[i][j]);
        printf("\n");
    }
}
```

1、串程序的执行

- fork 一个新进程
- 新进程 exec("matrixSquare",), 从main() 的第一条指令开始执行，直至程序运行结束。

2、串程序只能使用一颗CPU。没办法利用多核或多台计算机的运算能力加速程序的执行。

- 会出现一颗CPU运行很久，其余CPU空闲的情况。

3、一般而言，串程序只能使用一个外设。很难同时使用多个外设的数据传输服务。

- 一台外设IO结束前，进程同步等待，没办法向第二台外设发IO命令。

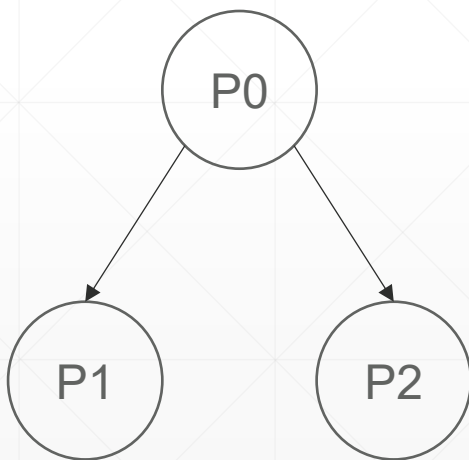
串行程序使用的地址空间（虚空间）

代码、只读数据
数据、bss
堆
用户栈

只有一个栈，所以程序只有一个
执行thread，只能用一颗CPU

2、进程级并行:

用fork 一个应用程序执行实例、多个进程



```
float A[n][n];  
float B[n][n];
```

```
int main( )
```

```
{
```

```
    初始化矩阵A, 清0矩阵B .....
```

```
    int pid = fork ( ) ;
```

```
    if (pid == 0)
```

```
        计算结果矩阵B的前半段; // 子进程P1
```

```
    else {
```

```
        pid = fork ( ) ;
```

```
        if (pid == 0)
```

```
            计算结果矩阵B的后半段; // 子进程P2
```

```
        else {
```

```
            wait( );
```

```
            wait( );
```

```
            输出结果矩阵 B; // 父进程 P0
```

```
        }
```

```
    }
```

```
}
```

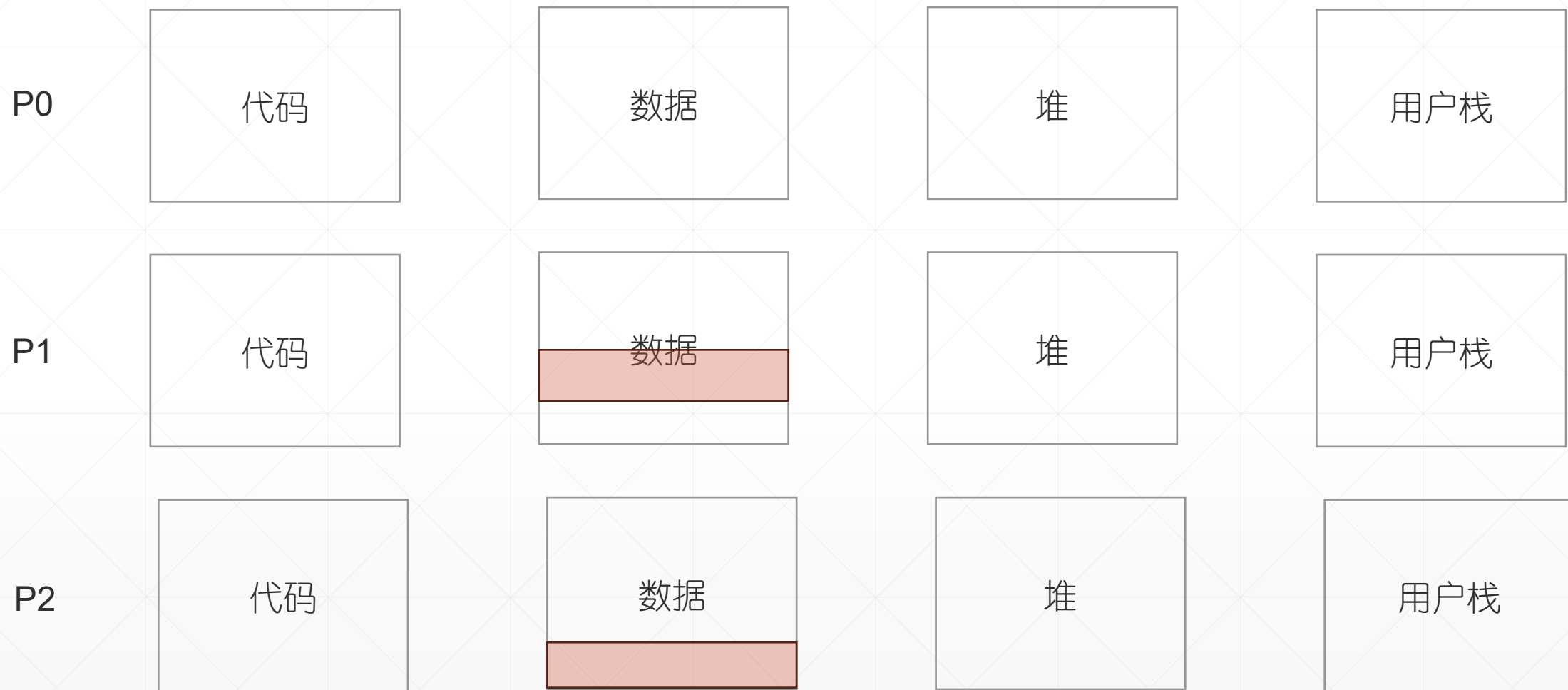
并行程序使用的虚空间 (进程级并行)



3个完整的进程虚空间，可以容纳更多的数据

3个栈，3个执行thread，应用程序可以用3个CPU

并行程序使用的虚空间（进程级并行）



存在的问题：进程的虚空间是隔离的。P0 取不到 P1、P2 的运算结果。

进程间通信机制 (单台计算机)

- 文件系统
 - 写文件。运算结果写文件。
 - 传消息。
 - socket (Unix域 套接字)
 - 消息队列
 - 管道 (无名管道PIPE, 有名管道FIFO文件)
- 共享内存
 - 共享内存 shared memory。
 - 物理内存创建共享内存 (shmget) , 一块装A矩阵, 另一块装B矩阵。
 - 所有进程将共享内存映射到自己的虚空间 (shmat) 。
 - 任何进程对矩阵元素的更新, 其它进程立即可见。
 - 文件内存映射 mmap。
 - 文件A装A矩阵、文件B装B矩阵
 - 所有进程将文件映射到自己的虚空间 (地址空间) ,
 - 用访问matrix[i][j]元素的方式读写文件 (无需借助 read、write 系统调用)
 - 任何进程对矩阵元素的更新, 其它进程立即可见。

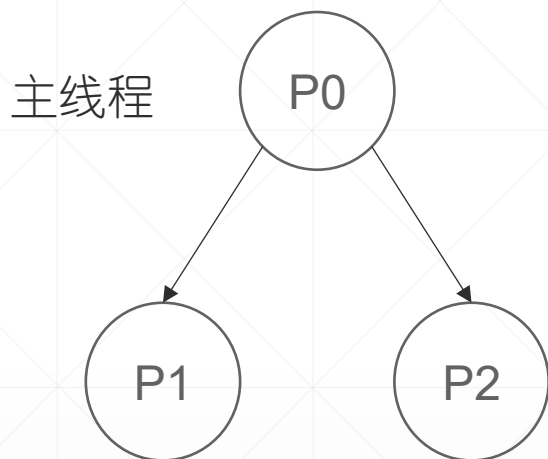


进程间通信机制（不同的计算机）

- socket (Internet域套接字)
 - 原始socket (TCP、UDP)
 - RPC (Remote Procedure Call, 远程过程调用)
 - MPI (Message Passing Interface, 消息传递接口)
- 借助文件服务器

2、线程级并行：

用pthread 一个应用程序执行实例、多个线程



./thread 1000

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

volatile int counter = 0;
int loops;

void *worker(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        counter++;
    }
    return NULL;
}
```

```
int main(int argc, char *argv[])
{
    if (argc != 2) {
        fprintf(stderr, "usage: threads <value>\n");
        exit(1);
    }

    loops = atoi(argv[1]);
    pthread_t p1, p2;
    printf("Initial value : %d\n", counter);

    Pthread_create(&p1, NULL, worker, NULL);
    Pthread_create(&p2, NULL, worker, NULL);
    Pthread_join(p1, NULL);
    Pthread_join(p2, NULL);
    printf("Final value : %d\n", counter);
    return 0;
}
```

代码注释

- thread是并程序。主线程 P0 从 main 函数入口开始运行，2次调用 Pthread_create 函数创建 peer线程 P1 和 P2。之后 Pthread_join 睡眠等待 线程P1、P2 终止。
- Pthread_create 函数 的第3个参数是一个子程序，新创建的线程执行该子程序。回到我们的例子，创建成功后，P1 和 P2 从 worker子程序的入口开始执行。子程序返回，线程终止，唤醒主线程，后者通过 Pthread_join 函数。
- 所以，在运算阶段，负责执行这个程序的有3个线程，P0、P1、P2。双核系统中，P0睡眠，P1、P2 运行，各使用1个CPU。CPU利用率 100%。

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
```

```
volatile int counter = 0;
int loops;
```

```
void *worker(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        counter++;
    }
    return NULL;
}
```

```
int main(int argc, char *argv[])
{
    if (argc != 2) {
        fprintf(stderr, "usage: threads <value>\n");
        exit(1);
    }

    loops = atoi(argv[1]);
    pthread_t p1, p2;
    printf("Initial value : %d\n", counter);

    Pthread_create(&p1, NULL, worker, NULL);
    Pthread_create(&p2, NULL, worker, NULL);
    Pthread_join(p1, NULL);
    Pthread_join(p2, NULL);
    printf("Final value : %d\n", counter);
    return 0;
}
```

并行程序使用的地址空间 (线程级并行)

P0、P1、P2 共享代码段、数据段 和 堆。
每个线程独用自己的栈。
所有线程共享同一个地址空间

共享数据存在数据段和堆中，
无需设置线程间通信机制

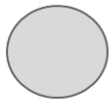


4、并行的好处

- CPU bound任务：获得更多CPU份额，缩短程序的运行时间。

```
int main()
{
    全部的计算量
}
```

串行程序



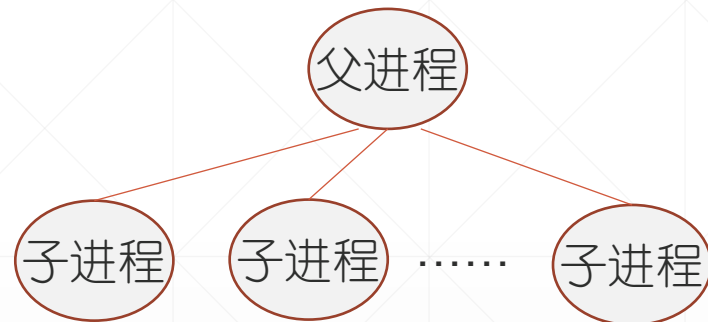
并程序

```
int main()
{
    .....
    int pid = fork ();
    if (pid == 0)
        子进程承担的计算
    else {
        父进程再创建子进程
        再创建子进程 .....
    }
}
```



9个参与运算的子进程（线程）

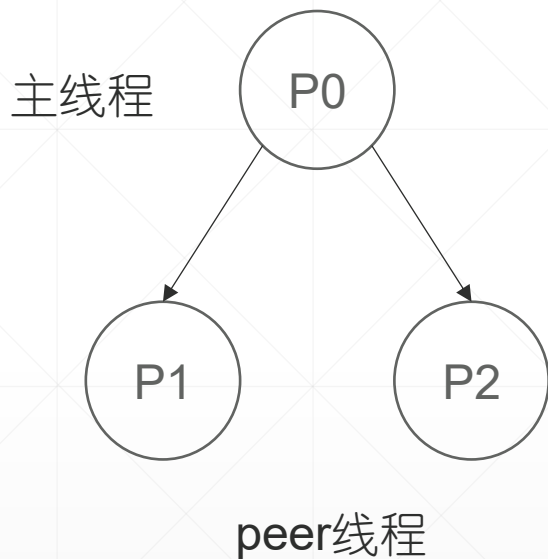
- IO bound任务（网络服务器）：多个IO通道并发，每个子进程服务一个通道，与一个client交互。



每个子进程监听一个socket，为一个client服务。它阻塞的时候，其余子进程还是活动的，可以通过另外的socket与其余client交互：

- 阻塞等待其它socket传来的数据
- 运算

5、并行，增加了程序设计的复杂度



`./thread 100000`

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
```

```
volatile int counter = 0;
int loops;
```

```
void *worker(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        counter++;
    }
    return NULL;
}
```

```
int main(int argc, char *argv[])
{
    if (argc != 2) {
        fprintf(stderr, "usage: threads <value>\n");
        exit(1);
    }

    loops = atoi(argv[1]); // 每个worker ++ 的次数
    pthread_t p1, p2;
    printf("Initial value : %d\n", counter);

    Pthread_create(&p1, NULL, worker, NULL);
    Pthread_create(&p2, NULL, worker, NULL);
    Pthread_join(p1, NULL);
    Pthread_join(p2, NULL);
    printf("Final value : %d\n", counter);
    return 0;
}
```

5、并行带来的麻烦





6、并发系统需要解决的重要问题

- 如何共享数据
- 互斥 (mutex):
保证共享变量的值不出错
- 同步(synchronization):
保证进程按照合理的时序运行、暂停。维护任务之间的前驱、后继关系

操作系统第五阶段 并发 和 进程通信

二、互斥、同步、信号量



1、并发访问时，共享变量的值为什么会出错（单核版）

- 现运行的应用程序PA执行完每条指令后，可能因为响应中断放弃CPU。
 - 下台时，保存所有用户态寄存器。被调度、再次运行时，恢复上次保存的用户态寄存器。
 - 运算期间，很多编译器会用寄存器存放共享变量副本。
 - 若PA在访问共享变量期间放弃CPU。。。放弃CPU期间若有其它进程更新共享变量的值。。。拿回CPU后，PA从恢复的寄存器得到的是旧值-----其它进程对变量的更新完全丢失，共享变量值出错。
- 本例：语句 `counter++`，经编译可能是3条指令：
1: `mov counter, %eax, ;` 2: `inc %eax;` 3: `mov %eax, counter`
 - T1时刻，`counter`变量值为2895，PA执行 `counter++` 操作，执行完指令1时间片到，响应时钟中断放弃CPU。 `eax==counter==2895`。
 - T2时刻，PA从指令2开始：++2895，赋值`counter`变量。。。
 - 若 (T1,T2) 时段，有进程访问、更新`counter`变量，无论执行过多少次`counter++`，更新无效。

2、怎样保证共享变量的值不出错

- **上锁**，保证共享变量的更新是原子操作。上例：counter++，一次完成。
- 并发进程互斥访问共享变量。用前上锁，用后解锁。访问期间，禁止其它进程的并发访问请求。

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
```

```
pthread_mutex_t m; 定义一个线程锁 m
volatile int counter = 0;
int loops;
```

上锁

```
void *worker(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        pthread_mutex_lock(&m);
        counter++;
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
```

解锁

```
int main(int argc, char *argv[])
{
    if (argc != 2) {
        fprintf(stderr, "usage: threads <value>\n");
        exit(1);
    }
```

```
    loops = atoi(argv[1]);
    pthread_t p1, p2;
```

初始化 m 开锁状态

```
    pthread_mutex_init(&m, NULL);
    pthread_create(&p1, NULL, worker, NULL);
    pthread_create(&p2, NULL, worker, NULL);
    pthread_join(p1, NULL);
    pthread_join(p2, NULL);
    printf("Final value : %d\n", counter);

    return 0;
}
```

相关概念、互斥，临界资源 和 临界区

- 互斥：一次只允许一个进程使用。
- 需要互斥保护的叫做临界资源，包括 共享变量（数据结构） 和 计算机系统中所有的外部设备。
- 临界区是访问共享变量的那段代码。

```
volatile int counter = 0;  
int loops;
```

临界资源

进入区，

entry section。上锁

临界区

退出区，exit section。解锁

```
void *worker(void *arg)  
{  
    int i;  
    for (i = 0; i < loops; i++) {  
        Pthread_mutex_lock(&m);  
        counter++;  
        Pthread_mutex_unlock(&m);  
    }  
    pthread_exit(NULL);  
}
```

2、锁的实现

- 互斥机制应遵循的原则（假设，进程 PA 需要访问共享资源 R）
 - 空闲让进：R空闲，可以使用
 - 忙则等待：R有进程用，等
 - 有限等待：不可以等太久
 - 让权等待：若要等待，入睡放弃CPU
(睡眠锁，适用于临界区很长或临界区内进程有可能入睡的情况)

2.1 软件实现的锁

```
int count = 0;    // 共享变量count
int lock = 0;    // 设置锁变量lock变量保护count变量。初值为0，锁是开的。
```

上锁

```
worker () { // P1
    while( )
    {
L1: while( lock==1 )
        ;
L2: lock = 1;

        counter++;

        lock = 0;
    }
}
```

临界区

解锁

上锁

```
worker () { // P2
    while( )
    {
        while( lock==1 )
            ;
        lock = 1;

        counter++;

        lock = 0;
    }
}
```

临界区

解锁

锁的使用过程:

现运行进程 P1 进 while 循环。

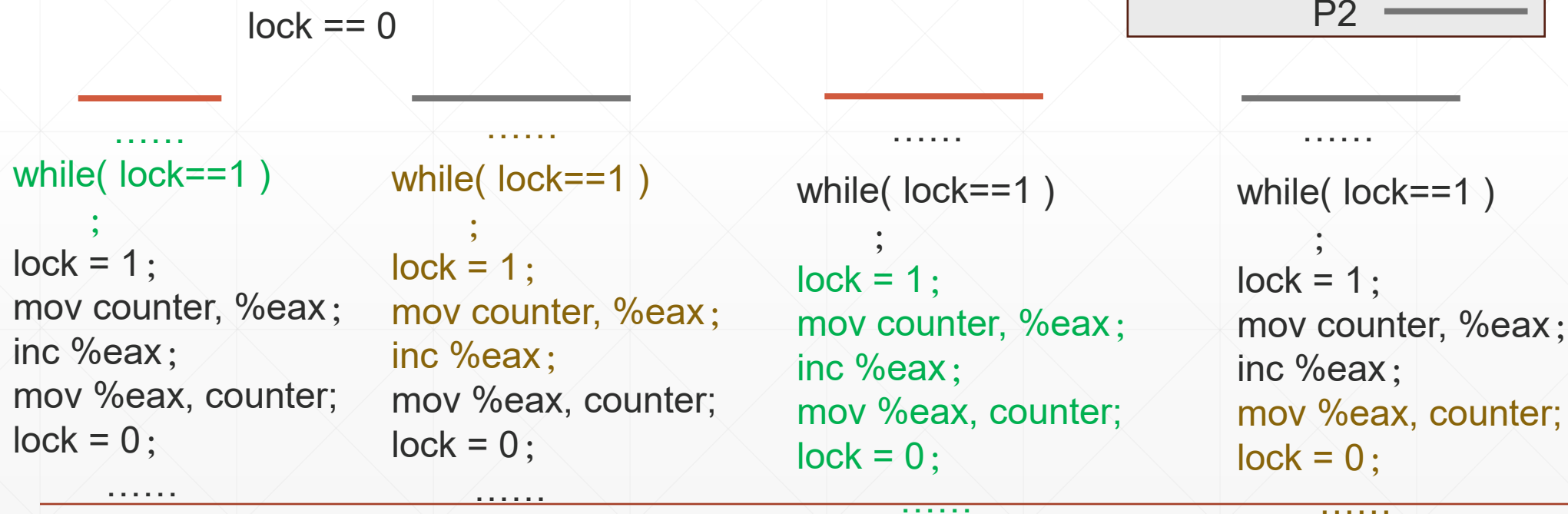
- lock==1, P2在用。忙等
 - lock==0, 上锁 lock=1
进临界区、counter++。
- 完成后, 解锁 lock=0。

这把锁大多数情况下能用, 因为P1在临界区的时候, 锁lock值为1, 可以阻止P2访问counter变量。

软件锁出错的情况

T1时刻，锁是开的。进程P1检测锁状态后，上锁（lock=1）前放弃CPU。

系统状态：临界区有P1进程、lock==0。出错。



软件锁没有保护好共享变量counter，P1 counter++更新丢失。

正确的锁机制必需保证：锁变量的检测和设置一次完成。

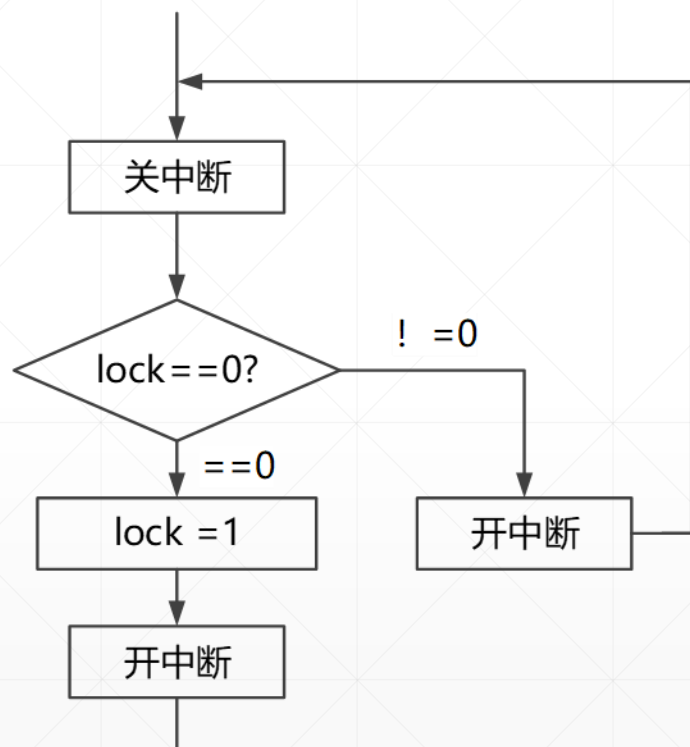


2.2 关中断

- 没有中断，就没有调度。

所以，内核可以用关中断的方法实现原子操作。

关中断实现的锁（硬件）



上锁操作

- 锁的检测和设置是原子操作，执行期间进程不会放弃CPU。

缺点：

- 关中断上锁，在多处理器系统中是无效的。无法阻止其它CPU访问同一个变量。
- 关中断时间必需足够短。这是因为，关中断时段，所有外设瘫痪，系统无法与外界交互，不能响应紧急任务。所以，只适合用它来实现原子操作。
- CLI, STI是操作系统的特权指令。APP无权使用。



附录 1：入睡为什么是原子操作？

```
void Process::Sleep(unsigned long chan, int pri)
{
    X86Assembly::CLI();
    this->p_wchan = chan; // 1
    this->p_stat = Process::SWAIT; // 4
    this->p_pri = pri; // 5
    X86Assembly::STI();
}
```

T时刻，P1执行sleep(5s)系统调用入睡，有进程P2定时器到期。时钟中断处理程序会把这2个进程全部唤醒。若不能保证入睡操作的原子性，允许唤醒和P1入睡操作并发，诸语句执行有可能是代码中高亮的次序，P1进程将会呈现：

```
this->p_wchan = 0;
this->p_stat = Process::SWAIT;
this->p_pri = pri;
```

的状态。永远不会被唤醒了。出错。

```
void Process::SetRun()
{
```

```
    ProcessManager& procMgr = Kernel::Instance().GetProcessManager();

    /* 清除睡眠原因，转为就绪状态 */
    this->p_wchan = 0; // 2
    this->p_stat = Process::SRUN; // 3
    if ( this->p_pri < procMgr.CurPri )
    {
        procMgr.RunRun++;
    }
    if ( 0 != procMgr.RunOut && (this->p_flag & Process::SLOAD) == 0 )
    {
        procMgr.RunOut = 0;
        procMgr.WakeUpAll((unsigned long)&procMgr.RunOut);
    }
}
```

2.3 应用程序可以使用的锁（硬件）

- 基于指令 XCHG(exchange)、CAS(compare and set) 或 TST(test and set)

```
int counter = 0;  
int lock = 0;
```

```
worker () { // P1  
    int gotLock;  
    while( )  
    {  
        gotLock = 1;  
        while( gotlock==1 )  
            xchg gotLock, lock ;  
  
        counter++;  
  
        lock = 0;  
    }  
}
```

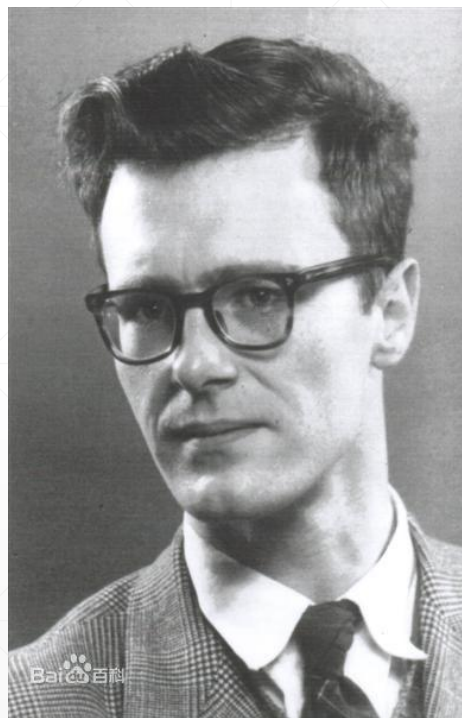
```
worker () { // P2  
    int gotLock;  
    while( )  
    {  
        gotLock = 1;  
        while( gotlock==1 )  
            xchg gotLock, lock ;  
  
        counter++;  
  
        lock = 0;  
    }  
}
```



以上，忙等锁

2.4 信号量 semaphore

荷兰学者, **Edsger Dijkstra**
1965年提出信号量机制



现代操作系统为应用程序提供的同步和互斥工具。

信号量的用法:

- 1、为共享资源 R 配一个信号量, 用来描述资源使用状态
semaphore semaphoreR;
- 2、进程访问共享资源前, 执行P操作, 同步等待资源访问权限。
- 3、访问结束后, 执行V操作, 释放资源访问权限、唤醒等待使用资源的另一个进程。

信号量是1个object

struct semaphore

```
{  
    int value; // 值, 可供使用的资源数量  
    struct sem_queue *head; // FIFO阻塞队列  
                                等待使用资源的进程  
} semaphore ;
```

value>0: R还有多少个资源可供使用。
value=0: R资源没了。阻塞队列为空。
value<0: value的绝对值表示阻塞队列中进程的数量。

1、初始化操作

信号量, 使用前初始化。

- 初始化value值
- 队列置null

信号量的P操作

void p(S)

```
{  
    S.value - = 1;  
    if(S.value < 0)  
        sleep( );  
}
```

2、

信号量的V操作

void v(S)

```
{  
    S.value += 1;  
    if(S.value < =0)  
        wakeup( );  
}
```

3、

P、V是原子操作

访问共享资源前执行P操作

```
信号量的P操作  
void p(S)  
{  
    S.value -= 1;  
    if(S.value < 0)  
        sleep( );  
}
```

检测、上锁、入队

共享资源使用完毕执行V操作

```
信号量的V操作  
void v(S)  
{  
    S.value += 1;  
    if(S.value <= 0)  
        wakeup( );  
}
```

解锁、修正系统状态，
唤醒等待使用共享资源的进程

操作系统保证 P、V操作的原子性

用信号量实现的互斥锁

- 互斥信号量初值为 1，又叫互斥锁。
- 例 1、保护累加器count变量

```
int count = 0 ;
```

```
semaphore mutex ;    //保护共享变量 count
```

```
mutex.value = 1 ;
```

```
...  
p(mutex);  
count++;  
v(mutex);  
...
```

```
...  
p(mutex);  
count--;  
v(mutex);  
...
```

- 例 2、队列操作

定义双向队列；

semaphore mutex ; //保护共享变量 count

mutex.value = 1 ;

入队

P (mutex) ;
入队操作 (4条语句)
V (mutex)

出队

P (mutex) ;
出队操作 (4条语句)
V (mutex)

3、同步

并发系统中，进程经常需要

- **观察系统状态，确定自己是不是可以继续前进。**
- **修正系统状态，维护系统状态一致性**

这种操作叫做同步。实施同步的位置，叫做同步点。

用信号量实施同步

设置同步信号量，维护系统状态。

在同步点上，

- 进程检测、更新系统状态，识别共享资源使用状态，必要时入睡、同步等待访问权限。核心是原子操作 **P**。
- 资源访问结束后，进程修正系统状态，释放资源访问权限、唤醒等待使用资源的其它进程。核心是原子操作 **V**。

信号量数据结构

```
struct semaphore
{
    int value;
    struct sem_queue *head;
} semaphore ;
```

同步 和 互斥的区别

打篮球，抢篮板球是**互斥**。

所有运动员独立、竞争篮球访问权。

```
semaphore mutex ;  
mutex.value = 1;
```

所有运动员

p(mutex)

抢球

v(mutex)

打羽毛球。同步。

羽毛球的使用权，在2位运动员之间轮转。

```
semaphore a, b ;  
a.value = 1;  b.value = 0;
```

运动员 a

运动员 b

P(a)

P(b)

打球

打球

V(b)

V(a)

信号量的初值，对系统正常运行至关重要。不同用途、应该设置不同的初值
互斥信号量，初值为1；同步信号量，初值编码资源分配逻辑（或时序）

例1： 司机和售票员之间的同步

司机负责开车、到站把车停稳。 售票员负责开关车门、保证乘客的安全

- 信号量初值: $\text{Drive} = 0;$ $\text{OpenDoor} = 1;$

司机:

```
While() {  
    P (Drive)  
  
    driving  
    Stop at next busStop  
  
    V (OpenDoor)  
}
```

售票员:

```
While() {  
    P (OpenDoor)  
  
    close the door  
    customer staging  
    selling tickets  
    close the door  
  
    V (Drive)  
}
```



优化：将 selling tickets 移到临界区 (P、V操作保护的代码片段) 以外

■ 信号量初值：

Drive = 0;

OpenDoor = 1;

司机：

```
While() {  
    P (Drive)  
  
    driving  
    Stop at next busStop  
  
    V (OpenDoor)  
}
```

售票员：

```
While() {  
    P (OpenDoor)  
  
    close the door  
    customer staging  
    close the door  
  
    V (Drive)  
    selling tickets  
}
```

这样，开车和售票就可以并发了。
会提升并发系统的性能。

driving不能移到临界区以外，为什么？